

Esame OS Internals (Cabodi)

Data: 20/06/2023

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, TLB e EAT)

Contesto: Sia dato un sistema di memoria virtuale con paginazione a richiesta, nel quale vengono indirizzati i Byte. Il sistema ha indirizzi su 32 bit, con pagine di 4KB, e dispone di TLB (Translation Look-aside Buffer). La tabella delle pagine ("page-table") è a un solo livello. Non si utilizzano ulteriori strutture dati (quali tabelle di hash o inverted page table) per velocizzare gli accessi. Un TLB miss viene gestito facendo accesso alla tabella delle pagine, gestendo l'eventuale PF, aggiornando la TLB, e ripetendo l'accesso in memoria con TLB aggiornata.

Si sa che la memoria RAM ha tempo di accesso di 100 ns, che un Page Fault ha un costo di 2.1 ms oppure 4.1 ms, a seconda del valore del modify bit relativo alla pagina scelta come vittima. Si sono valutate tre stringhe di riferimento w_1 , w_2 e w_3 , di lunghezza 100.000 (ognuna). In w_1 si osservano 10.000 TLB miss e nessun PF, in w_2 20.000 TLB miss e 10 PF (tutti con modify bit 0), in w_3 10.000 TLB miss e 20 PF, di cui 10 con modify bit 0 e 10 con modify bit 1. Si sa inoltre che le tre stringhe rappresentano il comportamento del sistema in esame, assumendo che w_2 e w_3 siano equiprobabili e con probabilità (sia w_2 che w_3) doppia rispetto a w_1 .

Si risponda alle seguenti domande:

-) Si calcoli il numero complessivo di accessi alla TLB (compresi gli accessi ripetuti dopo un miss) eseguendo, in sequenza, le tre stringhe di riferimento. Si calcoli quindi, ESCLUDENDO I TEMPI DI PF, il tempo effettivo di accesso in RAM (*Effective Access Time* - EAT), dapprima per la sequenza delle tre stringhe e poi per il sistema che rappresentano (tenendo conto delle relative probabilità).
-) Si considerino ora i PF. Si spieghi brevemente come va correlato il modify bit con i tempi di gestione dei PF. Si calcoli, per ognuna delle tre stringhe di riferimento e quindi per il sistema che rappresentano, il tempo effettivo di accesso in RAM, tenendo conto sia della TLB che dei PF.

Domanda 2 (File System e Allocazione Indicizzata)

Contesto: Considerare un file system basato su allocazione indicizzata nel quale si utilizza per ogni file una lista di blocchi indice. ATTENZIONE: non si tratta di organizzazione Unix-UFS basata su INODE, ma solo di lista di blocchi indice. I puntatori/indici hanno una dimensione di 32 bit e i blocchi del disco hanno una dimensione di 4KB. Il file system risiede su una partizione del disco in cui 2TB sono riservati a blocchi di dato e blocchi di indice, a cui si aggiungono 2GB riservati a metadati (FCB, direttori, ecc., blocchi indice esclusi in quanto allocati insieme ai blocchi dato).

-) Calcolare la massima dimensione possibile per un file (si vuole la dimensione esatta del file).

-) Dato un file binario di dimensione 8245 KB, calcolare quanti blocchi indice e blocchi di dato occupa il file. Calcolare anche la frammentazione interna, sia per i blocchi di dato che per quelli di indice.
 -) Si sa che il file (vedi domanda B) contiene una sequenza di numeri reali in formato `double` (64 bit per numero). Si calcoli fino a quale blocco di indice e a quale riga (casella) di questo occorre accedere per leggere i primi 512K (dove K va inteso come 2^{10}) numeri del file. (Verificare che la dimensione sia consistente).
-

Domanda 3 (Page Table e Bit di Stato)

Contesto: Si consideri un contesto di paginazione a richiesta con sostituzione di pagine. Si risponda alle domande seguenti:

-) Spiegare brevemente per quale/i scopo/i possono essere usati, nella Page Table, i bit denominati: *valid*, *reference*, *modify*.
-) Si tratta di bit presenti solo nella PT o anche nella TLB? (Motivare).
-) Si vuole realizzare una Page Table a un solo livello unica per tutti i processi. E' possibile? Se no, spiegare perché. Se sì, dire quale è il costo di una conversione pagina-frame.

SECTION 2: OS161

Domanda 4 (Thread e Processi in OS161)

Contesto: E' dato un sistema OS161.

-) Si considerino le funzioni `thread_switch` e `thread_fork`. Quale viene utilizzata per creare un processo? Quale per creare un nuovo kernel thread? (Motivare).
 -) Quando viene creato un nuovo kernel thread in OS161, questo appartiene al processo che lo ha creato o può appartenere a un diverso processo? (Motivare).
 -) Perché la funzione `thread_exit()`, nella versione base di OS161, non può essere chiamata dopo che si è chiamata `proc_destroy` (per il processo a cui appartiene il thread)?
 -) Cosa succede se non viene chiamata la funzione `as_destroy` al termine dell'esecuzione di un processo user? Il kernel può andare in crash? (Motivare).
-

Domanda 5 (Gestione Memoria `dumbvm` e Page Table)

Contesto: Si consideri, in OS161, la realizzazione della gestione della memoria virtuale. Si risponda alle seguenti domande:

-) Si supponga di voler realizzare in `dumbvm` la gestione dei frame liberi mediante free-list, invece che con bitmap (come fatto nella soluzione proposta per il lab 2).
- A1)** Si dica se la free list può essere una lista di frame oppure una lista di intervalli contigui di frame liberi. (Motivare).
- A2)** Qualora la lista fosse ordinata, andrebbe ordinata in base a indirizzi fisici, logici oppure in base alla dimensione di un intervallo libero? (Motivare).
-) Si supponga di voler invece realizzare una PAGE TABLE a livello di singolo processo. Viste le dimensioni ridotte della RAM fisica, si ipotizzi che una casella (entry) della PT occupi solo 2 Byte. Si ipotizzi di NON voler gestire un heap per il processo: il processo contiene quindi unicamente (come fatto in `dumbvm`) i segmenti di codice, di dati e lo stack. Si sa che:
 - Il segmento 1 (codice) richiede 4 pagine, inizianti all'indirizzo `0x400000`
 - Il segmento 2 (dati) richiede 3 pagine, inizianti all'indirizzo `0x500000`
 - Lo stack ha dimensione 18 pagine, che terminano, anziché al massimo indirizzo logico possibile (come fatto in `dumbvm`), all'indirizzo `0x3FFFFFFF`.
- B1)** Si calcoli la dimensione complessiva dell'address space e, al suo interno, il numero di pagine effettivamente usate (valide).
- B2)** Si calcoli quale sarebbe la dimensione di una page table organizzata mediante due page table standard distinte, una per i soli segmenti di codice e dato e una per lo stack.

ANSWERS

Domanda 1 (Memoria Virtuale, TLB e EAT)

Dati Iniziali:

- Dimensione stringhe (N): 100.000 riferimenti.
- Tempo accesso RAM: 100 ns.
- TLB Hit: 1 accesso RAM per il dato (100 ns).
- TLB Miss: 1 accesso RAM per la Page Table + 1 per il dato ($100 + 100 = 200$ ns).
- Costo PF: 2,1 ms (2.100.000 ns) per modify bit 0; 4,1 ms (4.100.000 ns) per modify bit 1.
- Probabilità: Essendo $P(w_2) = P(w_3) = 2 \times P(w_1)$ e la loro somma pari a 1, si deduce $P(w_1) = 0.2$, $P(w_2) = 0.4$, $P(w_3) = 0.4$.

A) Accessi alla TLB ed EAT (senza PF):

1. Numero complessivo di accessi alla TLB: Ogni riferimento genera almeno un accesso alla TLB. In caso di miss, la CPU gestisce l'errore e *ripete* l'accesso, generando un secondo accesso alla TLB per la medesima istruzione. Quindi, per ogni stringa, gli accessi totali alla TLB sono $N + \text{Numero_Miss}$:

- w_1 : $100.000 + 10.000 = 110.000$ accessi
- w_2 : $100.000 + 20.000 = 120.000$ accessi
- w_3 : $100.000 + 10.000 = 110.000$ accessi

Il numero complessivo per la sequenza è $110.000 + 120.000 + 110.000 = \mathbf{340.000}$ accessi.

2. EAT escludendo i Page Fault: La formula base per l'EAT (Effective Access Time) è: $EAT = T_{hit} + (P_{miss} \times \text{Penalità}_{PT})$, dove la penalità per leggere la tabella in RAM è 100 ns.

- $EAT(w_1) = 100 \text{ ns} + (\frac{10.000}{100.000} \times 100 \text{ ns}) = 100 + 10 = \mathbf{110 \text{ ns}}$.
- $EAT(w_2) = 100 \text{ ns} + (\frac{20.000}{100.000} \times 100 \text{ ns}) = 100 + 20 = \mathbf{120 \text{ ns}}$.
- $EAT(w_3) = 100 \text{ ns} + (\frac{10.000}{100.000} \times 100 \text{ ns}) = 100 + 10 = \mathbf{110 \text{ ns}}$.

EAT della Sequenza: $\frac{110+120+110}{3} = \mathbf{113,33 \text{ ns}}$.

EAT del Sistema (pesato): $(0.2 \times 110) + (0.4 \times 120) + (0.4 \times 110) = 22 + 48 + 44 = \mathbf{114 \text{ ns}}$.

B) Calcolo EAT includendo i Page Fault:

Correlazione Modify Bit: Il modify bit (o dirty bit) indica se la pagina è stata modificata (scritta) mentre si trovava in RAM. Se il bit è a 1 (attivo), il sistema operativo è obbligato a riscrivere il contenuto del frame vittima sul disco prima di potervi caricare la nuova pagina. Questo genera due operazioni di I/O sul disco (salvataggio vittima + lettura nuova pagina), che spiegano perché il tempo di gestione passi da 2,1 ms a 4,1 ms.

Calcolo dei nuovi EAT: Dobbiamo "spalmare" il costo totale in nanosecondi dei PF sull'intera stringa di 100.000 riferimenti per trovare il ritardo medio da sommare all'EAT di base.

- w_1 : 0 PF. Nessuna penalità. $EAT_{nuovo}(w_1) = \mathbf{110 \text{ ns}}$.
- w_2 : 10 PF con modify=0 (costo 2.100.000 ns l'uno).
Ritardo medio = $\frac{10 \times 2.100.000}{100.000} = 210 \text{ ns}$.
 $EAT_{nuovo}(w_2) = 120 \text{ ns (base)} + 210 \text{ ns (PF)} = \mathbf{330 \text{ ns}}$.
- w_3 : 10 PF con modify=0 (costo 2.100.000 ns) e 10 PF con modify=1 (costo 4.100.000 ns).
Ritardo medio = $\frac{21.000.000 + 41.000.000}{100.000} = \frac{62.000.000}{100.000} = 620 \text{ ns}$.
 $EAT_{nuovo}(w_3) = 110 \text{ ns (base)} + 620 \text{ ns (PF)} = \mathbf{730 \text{ ns}}$.

Nuovo EAT del Sistema (pesato):

$$(0.2 \times 110) + (0.4 \times 330) + (0.4 \times 730) = 22 + 132 + 292 = \mathbf{446 \text{ ns}}$$

Domanda 2 (File System e Allocazione Indicizzata)

Premessa sull'Architettura:

In una lista di blocchi indice, un blocco del disco (4 KB = 4096 Byte) viene interamente dedicato a memorizzare puntatori. Ogni puntatore occupa 32 bit (4 Byte).

Capacità totale del blocco = $4096/4 = 1024$ puntatori.

Essendo una *lista concatenata*, 1 puntatore in ogni blocco indice deve essere riservato per puntare al successivo blocco indice. Di conseguenza, i puntatori utili per mappare i blocchi dati sono **1023 per blocco indice**.

A) Calcolo della massima dimensione possibile per un file:

Il file system dispone di una partizione dati di $2 \text{ TB} = 2^{41} \text{ Byte}$.

Il numero totale di blocchi disponibili (sia per dati che per indici) è:

$$\text{Blocchi Totali} = \frac{2^{41}}{2^{12}} = 2^{29} = 536.870.912 \text{ blocchi}$$

Per massimizzare il file, raggruppiamo i blocchi in set logici: per ogni 1023 blocchi dati allocati, è strettamente necessario 1 blocco indice. Sia X il numero di blocchi indice massimi:

$$X + 1023X = 536.870.912 \implies 1024X = 536.870.912$$

$$X = \frac{536.870.912}{1024} = 524.288 \text{ blocchi indice}$$

I restanti blocchi sono blocchi dati:

$$\text{Blocchi Dati} = 524.288 \times 1023 = 536.346.624 \text{ blocchi}$$

La dimensione massima in Byte del file è:

$$\text{Dimensione Massima} = 536.346.624 \times 4096 \text{ B} = \mathbf{2.196.875.771.904 \text{ Byte}} (\approx 1,997 \text{ TB})$$

B) File binario da 8245 KB: blocchi e frammentazione:

1. Calcolo Blocchi Dati:

$$\text{Blocchi Dati} = \lceil \frac{8245 \text{ KB}}{4 \text{ KB}} \rceil = \lceil 2061,25 \rceil = \mathbf{2062 \text{ blocchi}}$$

2. Calcolo Blocchi Indice:

Sapendo che un blocco indice copre 1023 blocchi dati:

$$\text{Blocchi Indice} = \lceil \frac{2062}{1023} \rceil = \lceil 2,015 \rceil = \mathbf{3 \text{ blocchi indice}}$$

3. Frammentazione interna Dati:

Spazio allocato = $2062 \times 4 \text{ KB} = 8248 \text{ KB}$.

Spazio utilizzato = 8245 KB.

Frammentazione = $8248 - 8245 = 3 \text{ KB (3072 Byte)}$.

4. Frammentazione interna Indici:

I primi due blocchi indice sono saturi. Il terzo blocco indice mappa i restanti blocchi dati:
 $2062 - (2 \times 1023) = 2062 - 2046 = 16$ puntatori usati.

Essendo la capacità di 1023 puntatori dati, avanzano $1023 - 16 = 1007$ slot vuoti.

Frammentazione = $1007 \times 4 \text{ B} = 4028 \text{ Byte}$.

C) Ricerca e accesso ai primi 512K numeri (Double):

- Dimensione singolo dato: Double = 64 bit = 8 Byte.
- Quantità da leggere: $512\text{K} = 512 \times 1024 = 524.288$ numeri.
- Byte totali da leggere: $524.288 \times 8 \text{ B} = 4.194.304 \text{ Byte} = 4096 \text{ KB}$.
- Quantità di blocchi logici attraversati: $\frac{4096 \text{ KB}}{4 \text{ KB}} = 1024$ blocchi.

Per leggere un volume pari a 1024 blocchi esatti (numerate con indice da 0 a 1023), occorre attraversare le strutture d'indice:

- Il **Primo blocco indice** copre i primi 1023 blocchi dati (indici da 0 a 1022).
- Non è sufficiente. Occorre accedere all'ultimo blocco logico necessario, avente indice **1023**, il quale si trova mappato nel blocco indice successivo.
- Conclusione: Occorre accedere al **Secondo Blocco Indice**. La casella (riga) da leggere per trovare il blocco logico 1023 sarà esattamente la **prima riga (casella di indice 0)** di tale Secondo Blocco Indice.

Domanda 3 (Page Table e Bit di Stato)

A) Scopo dei bit nella Page Table:

- **Valid Bit:** Indica se la traduzione è valida, ovvero se la pagina logica è attualmente residente in un frame della memoria fisica (RAM). Se vale 0, l'accesso scatena un'eccezione hardware di Page Fault, delegando al Sistema Operativo il compito di recuperare la pagina dal disco (swap o file system).
- **Reference Bit:** Viene impostato a 1 dall'hardware ogniqualvolta la pagina subisce un *qualsiasi* tipo di accesso (lettura, scrittura o fetch di istruzioni). Viene utilizzato dal Sistema Operativo (che lo resetta periodicamente) per supportare gli algoritmi di Page Replacement (come LRU approssimato o Second-Chance/Clock), permettendo di individuare le pagine non utilizzate di recente, che sono le candidate ideali per lo sfratto.
- **Modify (Dirty) Bit:** Viene impostato a 1 dall'hardware quando un processo esegue un'operazione di *scrittura* all'interno della pagina. Durante la sostituzione delle pagine, indica al SO se il contenuto del frame è stato alterato rispetto alla sua copia su disco. Se vale 1, la pagina deve essere scritta su disco prima di essere rimpiazzata; se vale 0, può essere sovrascritta immediatamente senza fare I/O, risparmiando tempo.

B) Presenza dei bit nella TLB:

SI, questi bit sono presenti anche all'interno della TLB.

Motivazione: La TLB è una cache hardware velocissima interna alla MMU. Affinché i meccanismi di protezione e tracciamento funzionino senza penalizzare le prestazioni, l'hardware deve poter controllare e aggiornare i bit *modify* e *reference* in un singolo ciclo di clock. Se questi bit risiedessero solo nella Page Table in RAM, ogni operazione di scrittura richiederebbe un costoso e aggiuntivo accesso in memoria per aggiornare il bit.

Inoltre, il **valid bit** assume una semantica leggermente diversa ma cruciale: mentre nella PT indica l'assenza della pagina in RAM (causando un Page Fault software), nella TLB il *valid bit* a 0 indica semplicemente che la riga della cache TLB è vuota o invalida (causando un TLB Miss risolvibile via hardware o via software attraversando la PT).

C) Page Table a livello singolo per tutti i processi:

SI, è possibile, ed è nota come Inverted Page Table (Tabella delle Pagine Invertita).

Motivazione e Funzionamento: Invece di allocare un array indicizzato per Numero di Pagina Virtuale (VPN), l'Inverted Page Table è indicizzata per **Numero di Frame Fisico**. Le sue dimensioni non dipendono dallo spazio di indirizzamento virtuale, ma solo dalla quantità di RAM fisica installata, occupando quindi pochissima memoria. Poiché più processi condividono questa tabella, ogni entry deve necessariamente memorizzare anche l'identificatore del processo (PID o ASID) a cui appartiene la pagina, oltre al VPN.

Costo della conversione: Poiché l'array è ordinato per Frame Fisico, la traduzione da un VPN a un Frame richiederebbe teoricamente una scansione lineare (costo $O(F)$). Per evitare questo collo di bottiglia, la tabella viene sistematicamente associata a una funzione di Hashing basata su $\text{Hash}(\text{PID}, \text{VPN})$. Grazie a questa struttura, il **costo di una conversione pagina-frame scende a $O(1)$** (costante) nel caso medio, degradando linearmente solo per la risoluzione di limitate catene di collisioni.

Domanda 4 (Thread e Processi in OS161)

A) Ruolo di `thread_switch` e `thread_fork`:

- **thread_switch:** Non crea né thread né processi. È una routine di basso livello (tipicamente scritta in assembly) responsabile esclusivamente del *context switch*, ovvero il salvataggio dello stato della CPU (registri) del thread uscente e il ripristino di quello del thread entrante.
- **thread_fork:** Viene utilizzata per **creare un nuovo kernel thread**. Alloca la struttura `thread`, prepara il suo stack del kernel e lo inserisce nella coda dei thread pronti (ready queue).
- *Nota sulla creazione del processo:* Nessuna delle due crea, da sola, un processo utente. Un processo richiede una struttura dedicata (`struct proc`, creata via `proc_create`) a cui un thread appena creato con `thread_fork` viene associato.

B) Appartenenza del nuovo kernel thread:

Il nuovo kernel thread **può appartenere a un processo diverso** da quello che ha eseguito la chiamata.

Motivazione: La funzione `thread_fork` accetta esplicitamente come parametro un puntatore al processo a cui il thread deve essere agganciato (`struct proc *proc`). Un caso emblematico è l'implementazione della system call `sys_fork()`: il processo padre alloca una nuova struttura di processo per il figlio e poi chiama `thread_fork` passando come argomento tale struttura. Di conseguenza, il processo padre genera fisicamente un thread che apparterrà al processo figlio.

C) Perché non chiamare `thread_exit` dopo `proc_destroy`?

Si verificherebbe un errore fatale di **Use-After-Free** che causerebbe il crash del kernel (Kernel Panic).

Motivazione: Durante la `thread_exit`, il thread corrente ha bisogno di smontarsi dalle strutture del processo a cui appartiene, dereferenziando il proprio puntatore `curthread->t_proc` (chiamando funzioni come `proc_remthread`). Se la `proc_destroy` è già stata chiamata in precedenza, l'area di memoria allocata per quel processo è già stata restituita al sistema tramite `kfree()`. Accedervi risulterebbe in un tentativo di dereferenziare un *dangling pointer* (memoria non più valida).

D) Mancata chiamata ad `as_destroy` e possibile crash:

Se `as_destroy` non viene chiamata, si verifica un **memory leak** (perdita di memoria). La struttura `addrspace` e, soprattutto, tutte le **pagine fisiche di RAM** allocate per quel processo non verranno mai restituite alla *free list* o alla bitmap del sistema.

Il kernel può andare in crash? SÌ.

La memoria RAM fisica è una risorsa strettamente limitata. Se i processi continuano a terminare omettendo la pulizia dell'address space, il sistema esaurirà progressivamente i frame fisici liberi (situazione di *Out Of Memory* - OOM). A quel punto, le successive chiamate per allocare memoria (es. `alloc_kpages` o `getppages`) falliranno, potendo portare il kernel in *panic* o bloccando l'intero sistema.

Domanda 5 (Gestione Memoria `dumbvm` e Page Table)

A) Gestione dei frame liberi mediante free-list in `dumbvm`:

• A1) Lista di frame o intervalli contigui? Intervalli contigui.

Motivazione: L'implementazione di base di `dumbvm` non dispone di hardware avanzato di paginazione capace di mappare frame sparsi in un indirizzamento logico contiguo. Le allocazioni di sistema (es. tramite `getppages()`) richiedono che le porzioni di memoria fisica restituite siano rigorosamente contigue. Pertanto, la free-list deve gestire intervalli (blocchi) contigui di frame liberi. Inoltre, gestire intervalli permette una facile *coalescenza* (inglobamento) dei frame adiacenti al momento della deallocazione.

• A2) Criterio di ordinamento: Dimensioni o Indirizzi Fisici.

Motivazione: Non è possibile ordinare per indirizzi logici, in quanto i frame fisici liberi non appartengono ad alcuno spazio di indirizzamento logico. L'ordinamento dipenderà dall'obiettivo primario di progettazione:

- Ordinare per **dimensione dell'intervallo** ottimizza le politiche di allocazione come il *Best-Fit* o il *Worst-Fit*, permettendo di trovare il blocco desiderato in tempo utile.
- Ordinare per **indirizzo fisico crescente** ottimizza drasticamente l'operazione di coalescenza: quando un processo libera un frame, è banale controllare i nodi adiacenti nella lista per fonderli in un unico grande intervallo contiguo.

B) Page Table a singolo processo:

Ipotizziamo entry PT da 2 Byte e pagine da 4 KB (0x1000 Byte).

1. B1) Dimensione Address Space e Pagine valide:

L'Address Space (spazio logico totale indirizzabile) parte da 0 e si estende fino all'indirizzo massimo consentito allo stack, ovvero 0x3FFFFFFF. Convertendo tale valore, otteniamo una dimensione complessiva dell'address space di circa **1 GB** (esattamente 1.073.741.824 Byte).

Al suo interno, la memoria effettivamente allocata corrisponde alle sole **pagine valide**:

$$\text{Pagine Valide} = 4 \text{ (codice)} + 3 \text{ (dati)} + 18 \text{ (stack)} = \mathbf{25 \text{ pagine}}$$

L'effettiva occupazione di memoria in RAM è quindi di $25 \times 4 \text{ KB} = 100 \text{ KB}$.

2. **B2) Dimensione di due Page Table standard distinte:**

Una Page Table *standard* è un array indicizzato in cui la posizione corrisponde al Virtual Page Number (VPN). Non potendo omettere indici interni, la tabella deve rappresentare anche gli spazi vuoti tra i segmenti che copre.

- **PT1 (Codice e Dati):** Il segmento codice inizia all'indirizzo $0x400000$ (VPN: $0x400 = 1024$) e occupa 4 pagine (da 1024 a 1027). Il segmento dati inizia all'indirizzo $0x500000$ (VPN: $0x500 = 1280$) e occupa 3 pagine (da 1280 a 1282). L'array della PT1 deve spaziare dal VPN 1024 al VPN 1282 in modo ininterrotto.

$$\text{Numero Entry PT1} = 1282 - 1024 + 1 = \mathbf{259 \text{ entries}}$$

(Composte da 4 pagine di codice, 252 pagine "vuote" invalide, 3 pagine di dati).

$$\text{Dimensione PT1} = 259 \times 2 \text{ Byte} = \mathbf{518 \text{ Byte}}$$

- **PT2 (Stack):** Lo stack è un segmento compatto e ininterrotto di 18 pagine alla fine dello spazio di indirizzamento. La sua tabella dedicata richiederà esattamente un'entry per ogni pagina contigua:

$$\text{Numero Entry PT2} = \mathbf{18 \text{ entries}}$$

$$\text{Dimensione PT2} = 18 \times 2 \text{ Byte} = \mathbf{36 \text{ Byte}}$$